

SPRING BOOT 4 · NOVEMBER 2025

What's New in Spring Boot 4

Spring Framework 7 & Spring Boot 4



Dan Vega

Spring Developer Advocate · Broadcom

WHO'S TALKING

Dan Vega

Spring Developer Advocate · Broadcom

BACKGROUND

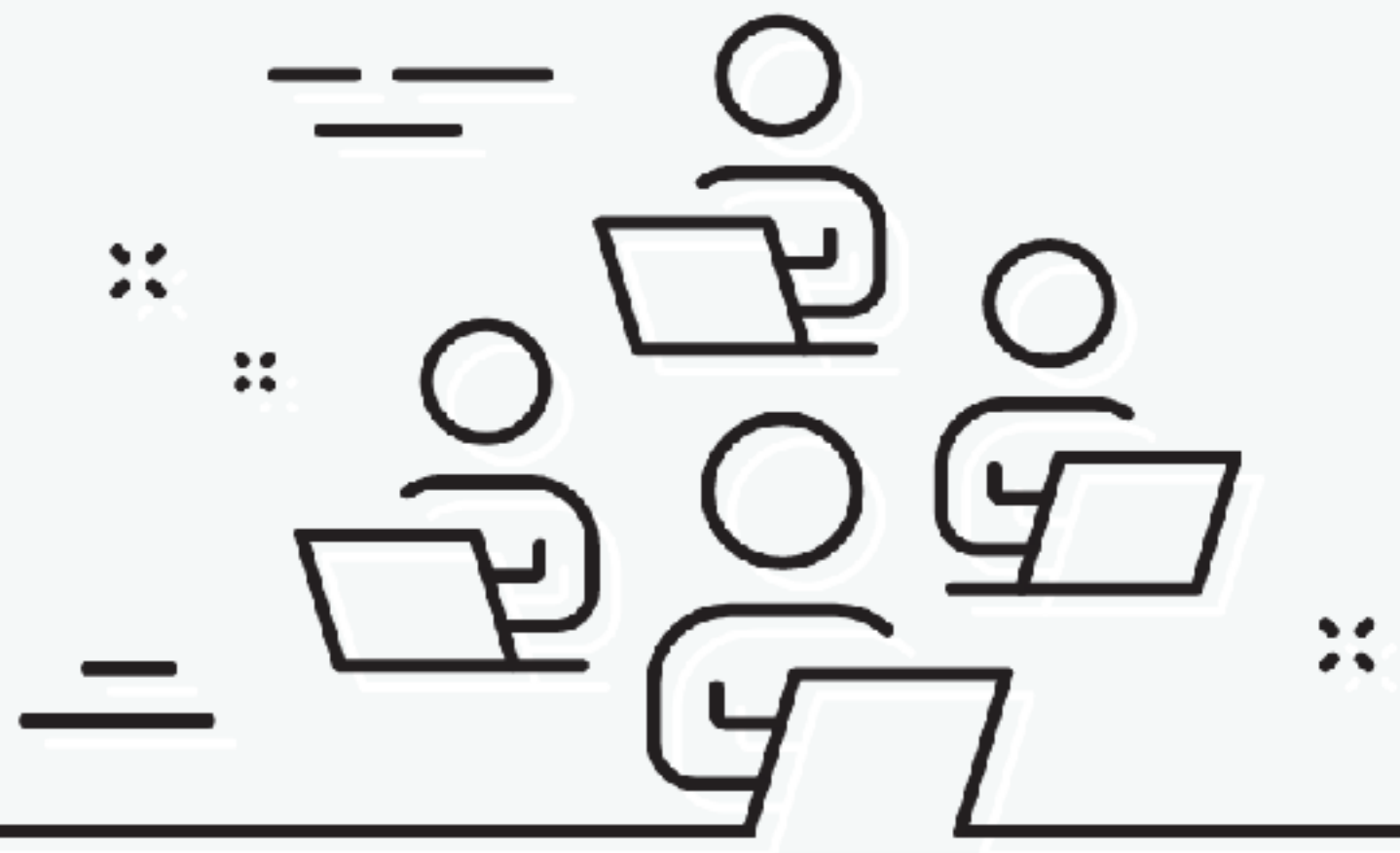
- Java Champion
- Author: *Fundamentals of Software Engineering* (O'Reilly)
- 25+ years building software
- Cleveland, OH · Husband & Father

FIND ME

- danvega.dev
- YouTube · Blog · Podcast
- X / Twitter: @therealdanvega
- LinkedIn, Bluesky



OFFICE HOURS



<https://www.springofficehours.io>

PART ZERO

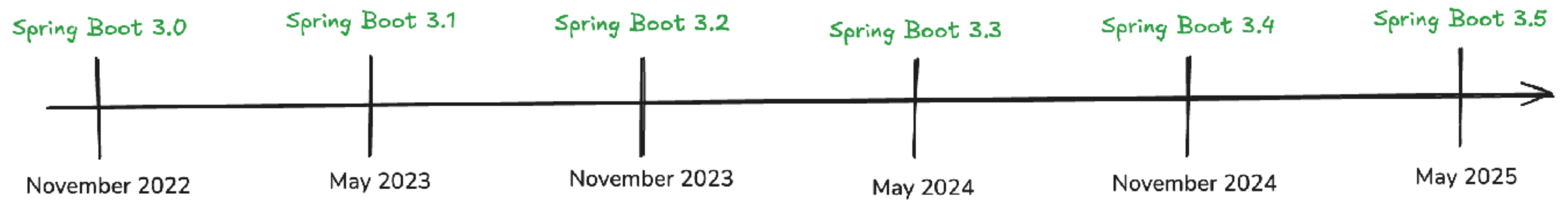
Where We've Been

A quick recap of the Spring Boot 3.x line.

“

You can't really know where you are going until you know where you have been.

— Maya Angelou



Three years, six releases

3.0 Nov 2022

JDK 17+ · Jakarta EE 9/10 · AOT & GraalVM ·
Observability · HTTP Interface Clients ·
Problem Details

3.1 May 2023

Docker Compose support · Testcontainers ·
Spring Authorization Server

3.2 Nov 2023

JDK 21 LTS · Virtual Threads · RestClient ·
JdbcClient · SSL bundle reloading

3.3 May 2024

CDS support · Observability upgrades ·
SBOM actuator · Service Connections

3.4 Nov 2024

Structured Logging · @Fallback beans ·
AssertJ for MockMvc · ARM images

3.5 May 2025

SSL bundle metrics · Properties from env vars
· Quartz from Actuator · ECS logging

NOVEMBER 2025

Spring Framework 7 & Spring Boot 4

The first major release in three years.

Follow along, and run the code

Road to GA blog series

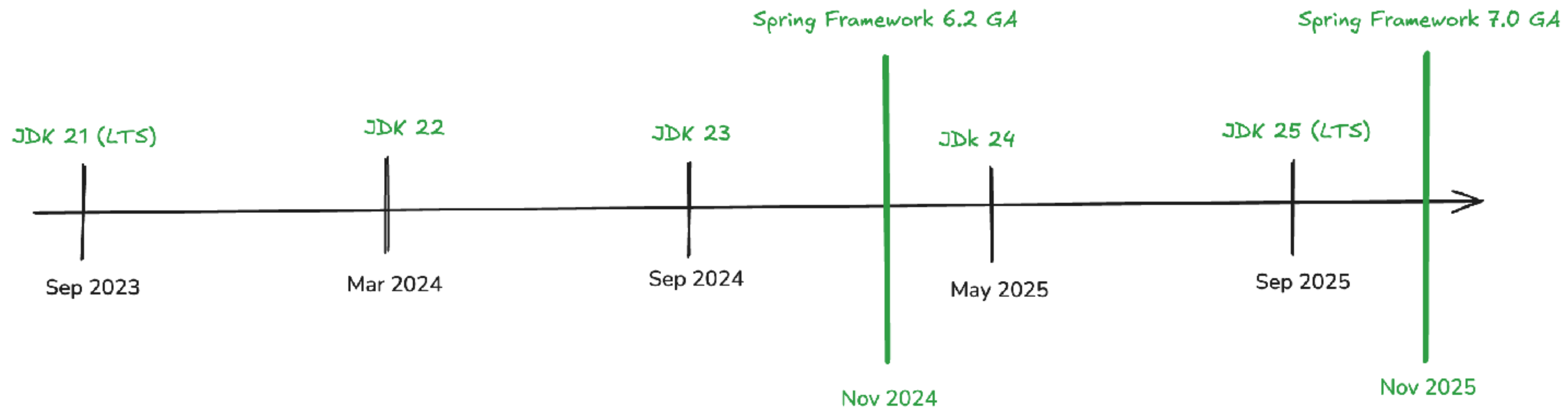
The Spring team documented every milestone on the way to 4.0.

spring.io/blog/2025/09/02/road_to_ga_introduction

Demo repository

Every example in this talk — runnable, in one place.

github.com/danvega/sb4



BASELINE UPGRADES

A fresh foundation under everything

JDK 17+ / JDK 25 LTS

Jakarta EE 11

Servlet 6.1 · Tomcat 11

JPA 3.2

Bean Validation 3.1

Hibernate ORM 7.x

Kotlin 2.2

GraalVM 25

Jackson 3

JUnit 6

Spring Boot 4 · Spring Framework 7

THE FOUNDATION

Modular Auto-Config

The biggest structural change in Spring Boot 4.

SPRING-BOOT-AUTOCONFIGURE JAR

185 KB → 2 MB

JAR growth from 2014 to 2025 — one monolith of auto-configuration.

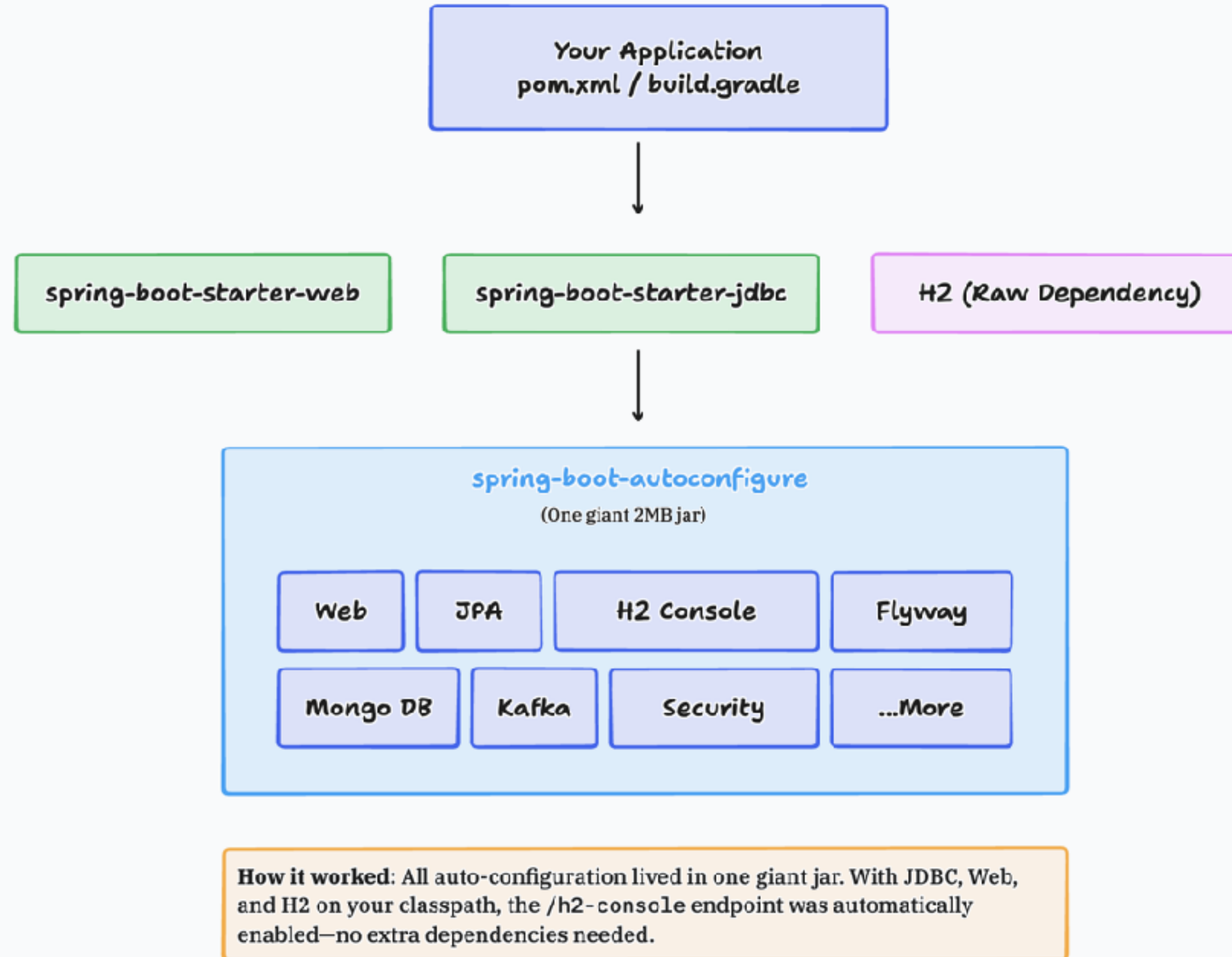
WHAT CHANGED

Auto-config split into focused modules

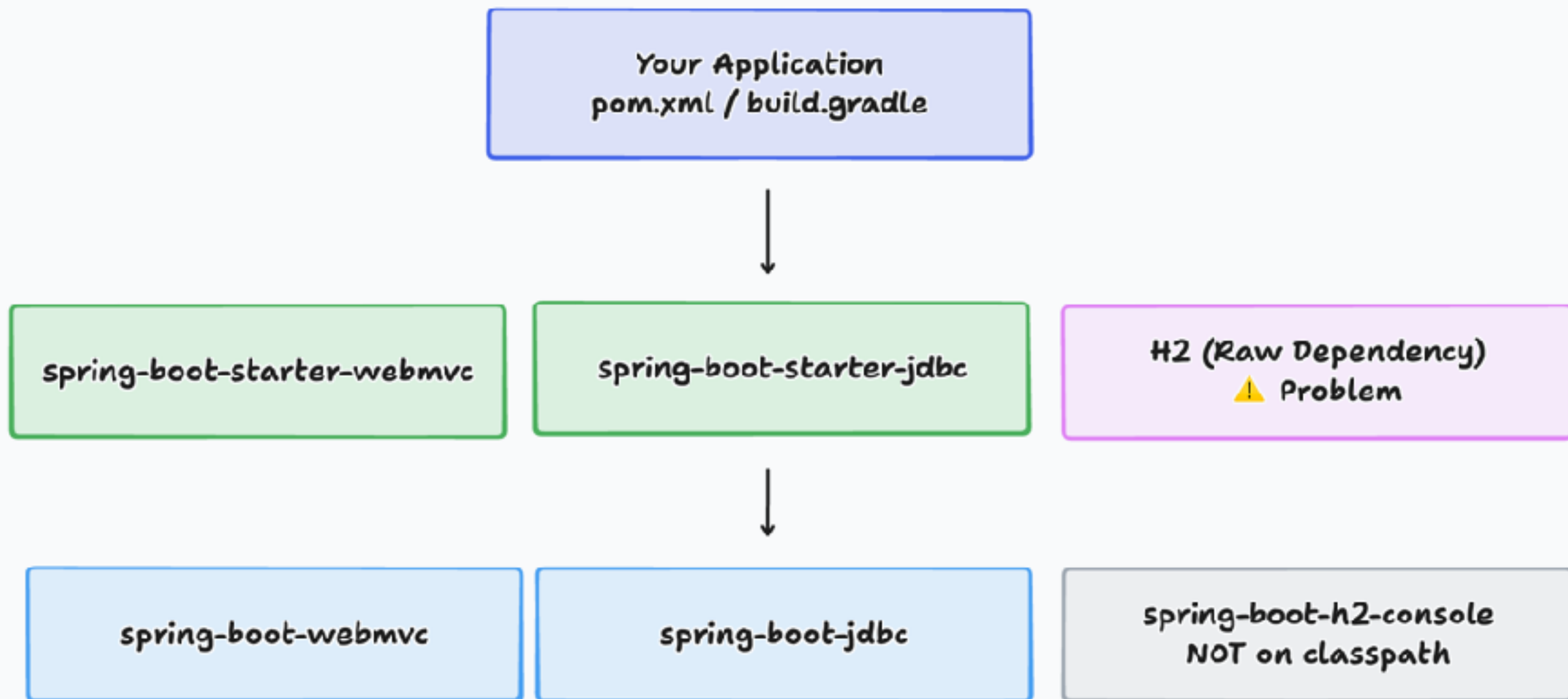
```
spring-boot-starter-web → spring-boot-webmvc
```

A new, more explicit `spring-boot-webmvc` starter replaces the catch-all. Each starter now brings **only its own** auto-configuration. Smaller footprint, sharper IDE auto-complete, fewer surprises on the classpath.

Spring Boot 3.x Architecture



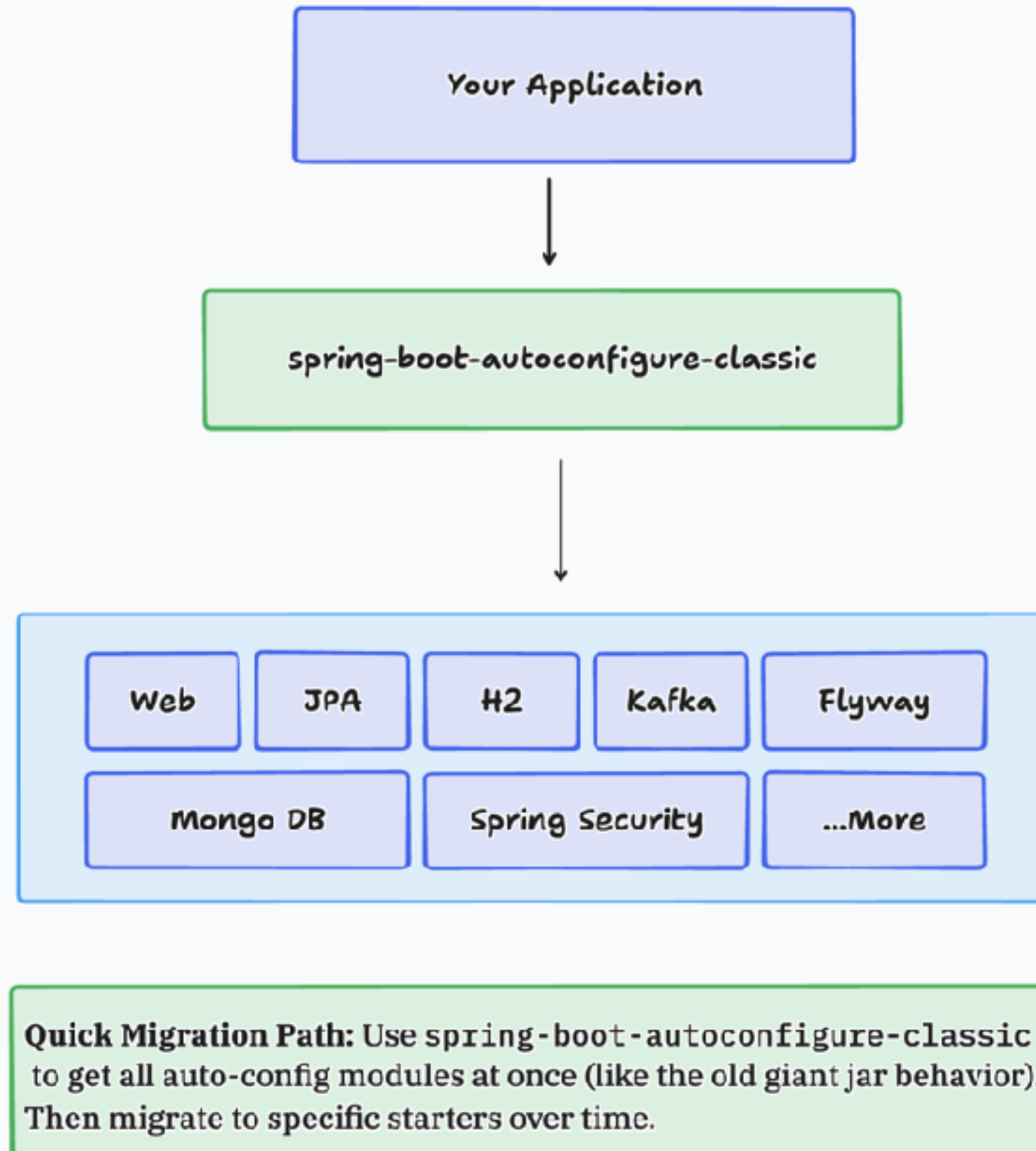
Spring Boot 4.x Architecture



Why H2 breaks: Auto-config code is now modular. You have the H2 dependency, but the auto-configuration code lives in `spring-boot-h2console` which isn't on your classpath!

The fix: Use `spring-boot-starter-h2console` which pulls in both the H2 library AND its auto-configuration module.

The Classic Escape Hatch



JSON, REWORKED

Jackson 3

A new JSON library with big improvements.

What's new in the box

- Immutable, builder-based configuration
- Unchecked exceptions, better for lambdas & streams
- Spring Boot auto-configures a `JsonMapper` bean
- ISO-8601 date defaults out of the box
- New `tools.jackson` packages
- Jackson 2 & 3 supported side by side during migration

Inject the auto-configured mapper

```
@Component public class DataLoader implements CommandLineRunner {

    private final JsonMapper jsonMapper;

    private final ResourceLoader resourceLoader;

    // constructor injection – Spring wires the JsonMapper bean
    public DataLoader(JsonMapper jsonMapper, ResourceLoader resourceLoader) {

        @Override
        public void run(String... args) throws Exception {
            Resource resource = resourceLoader.getResource(DONUTS_JSON_PATH);
            this .donuts = jsonMapper.readValue(resource.getInputStream(), new TypeReference <>() {});
        }

    }
}
```

Keep Jackson 2 defaults while you migrate

```
spring:
  application:
    name: donut-shop
  jackson:
    serialization:
      indent-output: true
    use-jackson2-defaults: true    # opt back in
```

Tag each field with the view it belongs to

```
public record Donut (  
    @JsonView(Views.Summary.class)  
    String type,  
    @JsonView(Views.Public.class)  
    Glaze glaze,  
    @JsonView(Views.Public.class)  
    List<String> toppings,  
    @JsonView(Views.Summary.class )  
    @JsonFormat(shape = Shape.STRING, pattern = "$#.##" )  
    BigDecimal price,  
    @JsonView(Views.Internal.class)  
    Integer calories,  
    @JsonView(Views.Internal.class)  
    LocalDateTime bakedAt  
) {}
```

Views compose by inheritance

```
public class Views {  
    // Minimal info for quick listings → type, price  
    public interface Summary {}  
  
    // Public API consumers → Summary + glaze, toppings, isVegan  
    public interface Public extends Summary {}  
  
    // Internal → Public + calories, bakedAt  
    public interface Internal extends Public {}  
  
    // Admin → everything, no restrictions  
    public interface Admin extends Internal {}  
}
```

From a mutable wrapper to a fluent hint

✗ Spring Boot 3

```
// mutable wrapper required
var user = new User("Marcel", ...);
var jv = new MappingJacksonValue (user);
// send wrapper, not object
jv.setSerializationView( Summary.class); var response = restTemplate.postForObject("/create" ,jv,String.class);
```

✓ Spring Boot 4

```
// clean, immutable, fluent
var user = new User("Marcel", ...);
var response = restClient.post()
    .uri( "/create" )
    .hint( JsonView.class .getName(),Summary.class )
    .body(user)      // the actual object
    .retrieve()
    .body(String.class);
```

WITH JSPECIFY

Null Safety

Say goodbye to NullPointerExceptions... maybe.

Ever hit a `NullPointerException`?

Yeah. Everyone. Every single one of you.

The problem isn't null

Sometimes you genuinely want to express the **absence of a value**.

The real problem is that nullness is usually **implicit and undocumented** and the compiler can't help you.

Four annotations, one standard

`@Nullable`

This type usage — field, return, parameter, generic — can be null.

`@NonNull`

Explicitly marks that null is not a valid value here.

`@NotNullMarked`

Everything in this scope is non-null by default.

`@NotNullUnmarked`

Nullness is unspecified — opt a scope back out.

ACROSS THE PORTFOLIO

JSpecify replaces Spring's JSR-305 annotations

~~@Nullable~~

~~@NonNull~~

~~@NonNullApi~~

~~@NonNullFields~~

Why are we doing this?

Ensure null safety in the IDE or during compilation time

Support for JSpecify

Across the Spring Portfolio of Projects

PROGRAMMATIC BEANS

BeanRegistrar API

A flexible way to register beans without abusing @Bean methods.

THE IDEA

Register multiple or conditional beans programmatically

Decide *which* beans to register at runtime. This can be driven by configuration, environment, or any logic you like.

Choose the implementation at registration time

```
public class MessageServiceRegistrar implements BeanRegistrar {
    @Override
    public void register(BeanRegistry registry, Environment env) {
        String messageType = env.getProperty("app.message-type", "email");

        switch (messageType.toLowerCase()) {
            case "email" → registry.registerBean("messageService",
                EmailMessageService.class,
                spec → spec.description("Email service via BeanRegistrar"));
            case "sms" → registry.registerBean("messageService",
                SmsMessageService.class,
                spec → spec.description("SMS service via BeanRegistrar"));
        }
    }
}
```

Import it like any other config

```
@Configuration
@Import(MessageServiceRegistrar.class)
public class ModernConfig {

    // other bean definitions here

}
```

BUILT-IN

API Versioning

First-class support in MVC & WebFlux.

THE SHIFT

We could always version our APIs

Now there's **first-class support** for web endpoint versioning in MVC and WebFlux. No more custom interceptors or filters.

WHAT YOU GET

A versioning toolkit, not just a flag

- `version` attribute on `@GetMapping` / `@PostMapping`
- Configurable strategy via `ApiVersionConfigurer`
- No more custom interceptors or filters
- Version source: header, param, media type or path
- Declare supported versions + a default version
- RFC 9745 deprecation hints, framework-emitted

Same path, two methods, one **version**

```
@GetMapping(value = "{version}/users", version = "1.0") public List<UserDT0v1 > findAllV1() {  
    return userRepository.findAll().stream().map(userMapper::toV1).toList();  
}  
  
@GetMapping(value = "{version}/users", version = "2.0") public List<UserDT0v2 > findAllV2() {  
    return userRepository.findAll().stream().map(userMapper::toV2).toList();  
}
```

Pick a strategy, configure once

```
@Configuration public class WebConfig implements WebMvcConfigurer {

    @Override
    public void configureApiVersioning(ApiVersionConfigurer c) {
        c
            .addSupportedVersions( "1.0", "1.1", "2.0" )
            .setDefaultVersion( "1.0" )
            .useRequestHeader( "X-API-Version" )
            .setVersionParser( new ApiVersionParser () );
    }
}
```

DECLARATIVE HTTP

HTTP Interface Clients

Declarative HTTP with `@HttpExchange`.

Introduced in Spring Framework 6 & Spring Boot 3.0

Spring Boot 4 makes them simpler. Still declare an interface, and Spring generates the client.

Describe the API as an interface

```
@HttpExchange(url = "/todos", accept = "application/json")
public interface TodoService {

    @GetExchange("/")
    List<Todo> getAllTodos();

    @GetExchange("/{id}")
    Todo getTodoById(@PathVariable Long id);

    @PostExchange("/")
    Todo createTodo(@RequestBody Todo todo);

}
```

No proxy factory boilerplate. Just import it.

```
@Configuration(proxyBeanMethods = false)
@ImportHttpServices(TodoService.class)
public class HttpClientConfig {

    // That's it!

}
```

Configuration

```
@Configuration
@ImportHttpServices(group = "jsonplaceholder", types = {TodoService.class, PostService.class})
@ImportHttpServices(group = "github", types = {RepoService.class, IssueService.class})
public class MultiApiConfig {

    @Bean
    RestClientHttpServiceGroupConfigurer groupConfigurer() {
        return groups → {
            groups.filterByName("jsonplaceholder")
                .forEachClient((group, builder) → builder
                    .baseUrl("https://jsonplaceholder.typicode.com/")
                    .build());

            groups.filterByName("github")
                .forEachClient((group, builder) → builder
                    .baseUrl("https://api.github.com")
                    .defaultHeader("Accept", "application/vnd.github.v3+json")
                    .build());
        };
    }
}
```

BUILT-IN

Resilience

No more Spring Retry dependency.

Retry & throttling, built into the core

- `@Retryable` — retry failed methods with backoff
- `@ConcurrencyLimit` — throttle concurrent calls
- Jitter support — prevent the thundering herd
- `RetryTemplate` — dynamic, programmatic control
- Exponential backoff (1s, 2s, 4s, 8s...)
- All built into Spring Framework 7 core

Retry a flaky call with one annotation

```
@Service
public class RestaurantService {

    @Retryable (maxAttempts = 4 ,includes = RestaurantApiException.class ,delay = 1000 ,multiplier = 2 )
    public List<MenuItem> getMenuFromPartner(String id) {
        // calls a flaky partner API...
    }

}
```

Resilience in Action

```
public DriverAssignmentService(DriverRetryListener driverRetryListener) {  
  
    this.driverRetryListener = driverRetryListener;  
  
    RetryPolicy retryPolicy = RetryPolicy.builder()  
        .maxAttempts(10)  
        .delay(Duration.ofMillis(2000))  
        .multiplier(1.5)  
        .maxDelay(Duration.ofMillis(10000))  
        .includes(NoDriversAvailableException.class)  
        .build();  
  
    retryTemplate = new RetryTemplate(retryPolicy);  
    retryTemplate.setRetryListener(driverRetryListener);  
}
```

Concurrency Cap

```
@Service
public class RestaurantNotificationService {

    private static final Logger log = LoggerFactory.getLogger(RestaurantNotificationService.class);

    @ConcurrencyLimit(3)
    public void notifyRestaurant(Order order) {
        LocalDateTime start = LocalDateTime.now();
        log.info("[CONCURRENT] Sending notification to restaurant for order {} (Thread: {})",
            order.id(), Thread.currentThread().getName());

        // Simulate notification taking time (network call, webhook, etc.)
        simulateDelay(Duration.ofSeconds(2));

        LocalDateTime end = LocalDateTime.now();
        log.info("[CONCURRENT] Notification sent for order {} (took {}ms)",
            order.id(), Duration.between(start, end).toMillis());
    }
}
```

A MODERN TESTING FACADE

RestTestClient

The WebClient API you know — for Spring MVC.

One fluent, AssertJ-style client for testing

It reads like a sentence, supports typed bodies and records, and is **versioning-aware** out of the box.

Method	Speed	What It Tests	Best For
<code>bindToController</code>	 Fastest	Just your controller	Quick unit tests
<code>bindToMockMvc</code>	 Fast	Controller + Spring MVC	Testing validation, security
<code>bindToApplicationContext</code>	 Slower	Full app (no HTTP)	Real database tests
<code>bindToServer</code>	 Slowest	Everything with HTTP	Complete end-to-end tests
<code>bindToRouterFunction</code>	 Fastest	Functional endpoints	WebFlux functional routes

Assertions that read like a sentence

```
@WebMvcTest(TodoSimpleController.class)
@AutoConfigureRestTestClient
public class TodoSimpleControllerTest {

    @Autowired RestTestClient client;

    @Test
    public void findAllTodos() {
        List<Todo> todos = client.get()
            .uri("/api/todos/simple/")
            .exchange()
            .expectStatus().isOk()
            .expectBody(new ParameterizedTypeReference<>() {})
            .returnResult().getResponseBody();

        assertEquals(1, todos.size());
    }
}
```

OBSERVABILITY

OpenTelemetry

A new Spring Boot starter.

KEY CONCEPTS

One dependency, production-ready

Single dependency `spring-boot-starter-opentelemetry` replaces the complex setup

Auto-instrumentation HTTP server / client, JDBC & more — out of the box

Log correlation Automatic trace / span ID injection into your logs

OTLP export Works with any OpenTelemetry-compatible backend

Production ready Official Spring support! No alpha dependencies

THE LGTM STACK

Where your telemetry lands

L

Loki

Log aggregation

G

Grafana

Dashboards & visualization

T

Tempo

Distributed tracing

M

Mimir

Long-term metrics storage

Point traces, metrics & logs at a collector

```
management:
  tracing:
    sampling.probability: 1.0      # 100% for dev
  otlp:
    metrics.export.url: http://localhost:4318/v1/metrics
  opentelemetry:
    tracing.export.otlp.endpoint: http://localhost:4318/v1/traces
    logging.export.otlp.endpoint: http://localhost:4318/v1/logs
```

Data source

Tempo ▾

Need help ▾

<<

Last 30 minutes ▾

 >>

🔍

🔄

Filters

Root spans

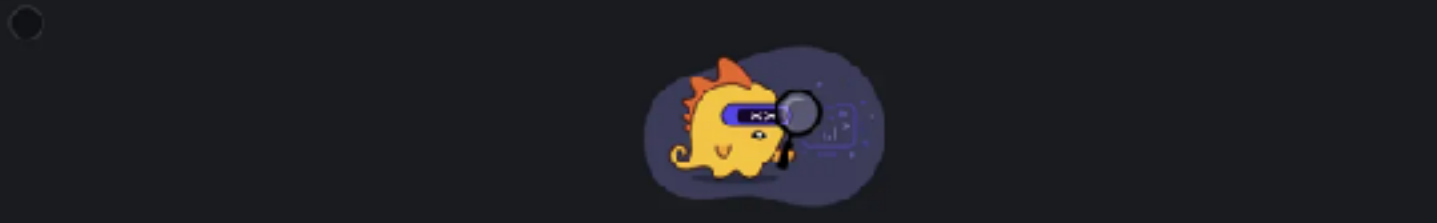
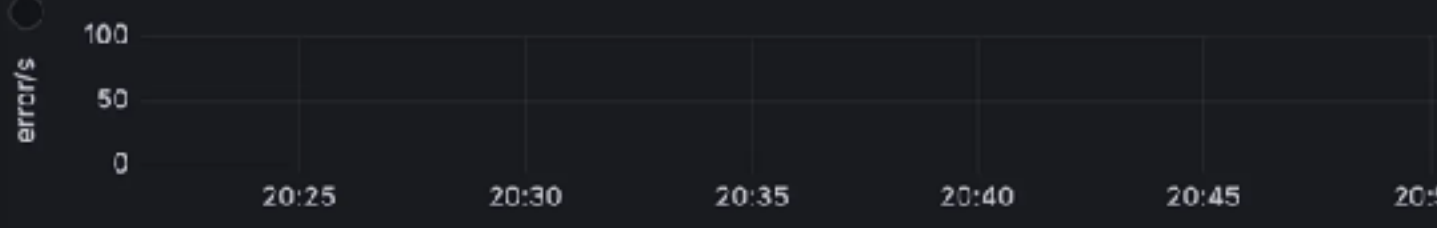
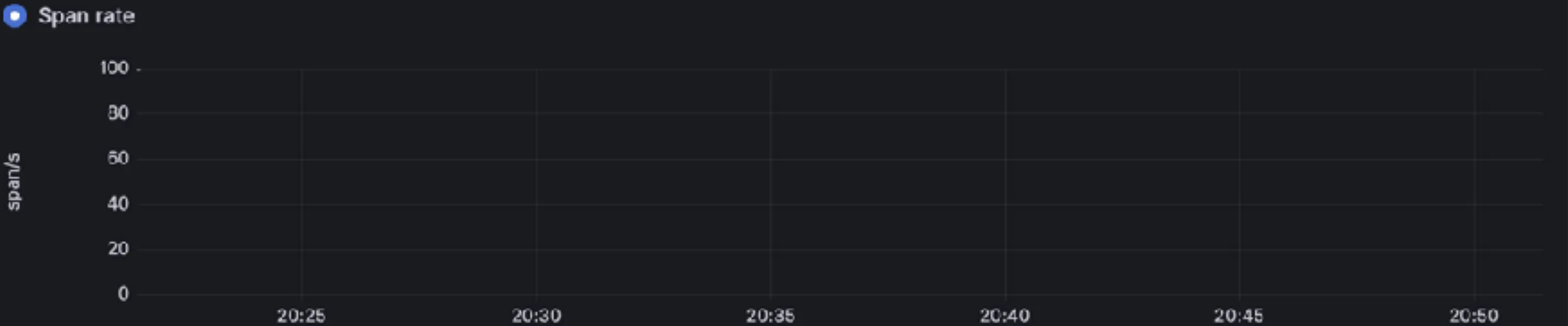
All spans ▾

🔍 Filter by label values

Trace ID

Enter an ID and press Enter

Select metric type ⓘ



Breakdown Service structure Comparison Traces 3

View a list of spans for the current set of filters.

Attributes

Selected (0): None

🔍 Search attributes...

Favorites

All

Resource

Span

☐

resource.

service.name

★

☐

span.

name

★

☐

span.

kind

★

☐

span.

rootName

★

☐

span.

rootServiceName

★

☐

span.

status

★

☐

span.

statusMessage

★

Start time	Trace Service	Trace Name	Duration	traceIdHidden	Name
2025-12-23 20:51:22.869	ot	http get /slow	502 ms	2ed6eaabb5c63f35d176c3eba	
2025-12-23 20:51:21.577	ot	http get /g http get /slow	60.0 ms	d79ccda3f596c9bb12cc804e1	
2025-12-23 20:51:20.145	ot	http get /	16.6 ms	1eaa080754c6abc64f5c345d	

MESSAGING, MODERNIZED

JMS Client

A fluent unified client, in the RestClient family.

UNIFIED JMS CLIENT

A modern alternative to JmsTemplate

- Fluent API, like `RestClient` & `JdbcClient`
- Unified exception translation
- Customizable QoS settings
- Supports `jakarta.jms` and Spring messaging

Send a message in two fluent lines

```
@Service
public class OrderMessagingService {
    \
    private final JmsClient jmsClient;

    public OrderMessagingService(JmsClient jmsClient) {
        this .jmsClient = jmsClient;
    }

    public void sendOrder(Order order) {
        jmsClient.send("orders.queue" ).withBody(order);
    }
}
```

PROJECT LOOM

Virtual Threads

Loom is landing across the JDK and Spring is ready.

Maturing release by release

JDK 21

Virtual Threads arrive

JDK 24

No more pinning issues

JDK 25

Scoped Values

JDK 27

Structured Concurrency?

One property turns it on

```
spring.threads.virtual.enabled=true
```

Controllers, `RestClient` , schedulers and listeners all benefit. In 4.0, Spring uses the JDK's virtual-thread executor under the hood wherever appropriate.

BEYOND THE FRAMEWORK

The Spring Portfolio

AI, gRPC, Data and Security move in lockstep with Boot 4.

GENERATIVE AI

Spring AI

Integrating Generative AI into your Spring applications.

We're consumers of models, not trainers

VERSIONS

Spring Boot 3 → Spring AI **1.1.5**

Spring Boot 4 → Spring AI **2.0.0** (Now GA!)

THE JAVA OPPORTUNITY

As application developers we aren't training models, we're consuming them. Spring AI is far more than a facility for making REST calls.



<https://www.danvega.dev/blog/can-you-use-java-for-ai>

GOOGLE REMOTE PROCEDURE CALL

Spring gRPC

A new first-class Spring Boot module (1.1.0 M1).

Why teams reach for gRPC

High performance

Binary protocol over HTTP/2 — lower latency than REST.

Strong typing

Protocol Buffers give type safety across services.

Streaming support

Bidirectional streaming out of the box.

Spring integration

Familiar auto-config, DI and annotations.

SPRING DATA

AOT Repositories

Generate repository code at build time.

The query methods you already write

```
@Repository
public interface CoffeeRepository extends ListCrudRepository<Coffee, Long> {

    List<Coffee> findByNameContainingIgnoreCase(String name);

    List<Coffee> findBySizeAndPriceGreaterThan(Size size, BigDecimal price);

}
```

THE PAYOFF

50–70%

faster startup with AOT repositories

Query parsing happens at build time, not runtime.

SPRING SECURITY

Multi-Factor Authentication

First-class MFA, built into Spring Security.

`MULTI-FACTOR AUTH`

First-class, and configurable

- `@EnableMultiFactorAuthentication`
- Global or selective, per-endpoint MFA
- Factor tracking via `FactorGrantedAuthority`
- PASSWORD + One-Time Token out of the box

WHAT IS MFA

Combine something you know, have, or are

Know: password, PIN

Have: SMS, email, token

Are: biometrics

Where: geolocation

Do: behavior profiling

THE SPRING SECURITY APPROACH

At authentication time, Spring adds a `FactorGrantedAuthority` per verified factor. Authorization rules then require multiple factors

e.g. `FACTOR_PASSWORD + FACTOR_OTT`.

Require both factors everywhere

```
@Configuration @EnableWebSecurity(debug = true)
@EnableMultiFactorAuthentication (authorities = {
    FactorGrantedAuthority .PASSWORD_AUTHORITY,
    FactorGrantedAuthority.OTT_AUTHORITY })
class SecurityConfig {

    @Bean
    SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        return http
            .authorizeHttpRequests(a → a
                .requestMatchers( "/", "/ott/sent" ).permitAll()
                .requestMatchers( "/admin/**" ).hasRole("ADMIN" )
                .anyRequest().authenticated())
            .formLogin(withDefaults())
            .oneTimeTokenLogin(withDefaults()).build();
    }
}
```

💡 Smart Redirect: Automatically sends user to missing factor's login

Demand MFA only where it matters

```
@Bean
SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
    var mfa = AuthorizationManagerFactories .multiFactor().requireFactors(
        FactorGrantedAuthority .PASSWORD_AUTHORITY,
        FactorGrantedAuthority .OTT_AUTHORITY)
        .build();

    http.authorizeHttpRequests(a → a
        .requestMatchers( "/admin/**").access(mfa.hasRole("ADMIN" ))
        .requestMatchers( "/user/settings/**" ).access(mfa.authenticated())
        .anyRequest().authenticated());

    return http.build();
}
```

/admin/** → Requires MFA + ADMIN role

/user/settings/** → Requires MFA only

Everything else → Single factor OK

Spring Boot 4.1 · May 21, 2026

gRPC first-party server / client / test modules, BOM-managed, same cadence as Boot

Type Safe Paths Introduces type-safe property paths in Spring Data Commons

SSRF protection `InetAddressFilter` blocks outbound calls to disallowed addresses

Async propagation observability context follows `@Async` automatically

Lazy JDBC `connection-fetch=lazy` defers the physical connection until a statement runs

START.SPRING.IO

Spring Boot 4.1.0 is available now

Generate a project, pick Java 26, and start building.

DEMO TIME

Show Me The Code

github.com/danvega/sb4

PART ZERO

Spring & Security in the times of AI

How a flood of AI-generated security reports is reshaping open source, and why Spring users have nothing to panic about.

BUT THERE IS A REAL STORY UNDERNEATH

AI made finding vulnerabilities cheap.

Code-scanning models have collapsed the skill and effort needed to surface a potential flaw. The result is a flood of security reports hitting open-source projects, all at once.

SPRING, APRIL 2026

482

new security reports in a
single month

≈ 6.5

historic average per month

65

projects scanned

26

new CVEs announced

AND SPRING IS NOT ALONE

The whole ecosystem is adjusting at once

MOZILLA

270+

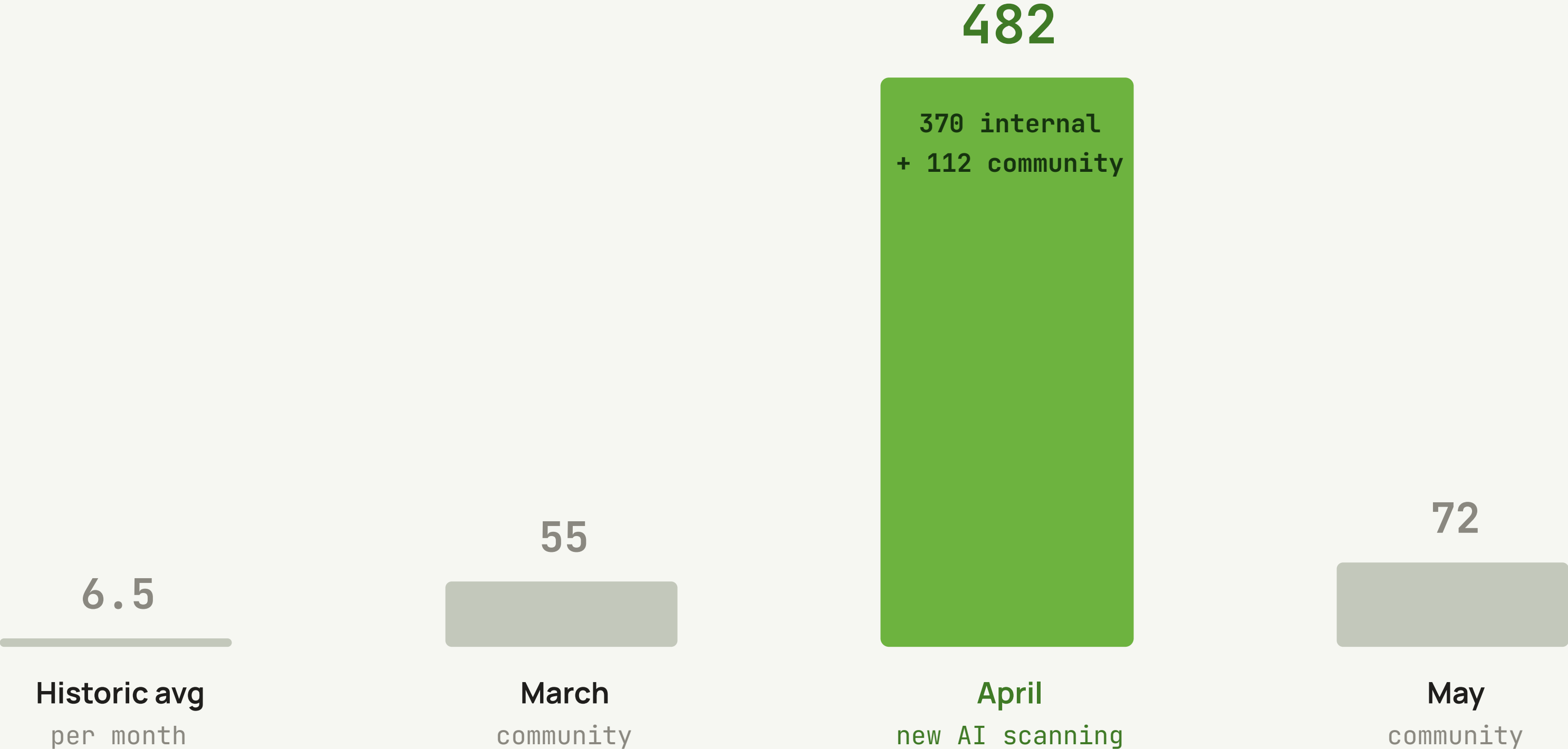
vulnerabilities fixed in Firefox 150, surfaced by an AI code-scanning preview.

FREEBSD

20-yr

old CVE in one of the industry's most secure operating systems, found via AI.

Reports by month



VOLUME IS NOT THE SAME AS RISK

Not every report is a CVE

37%

of internal scan results were
duplicates or invalid findings,
filtered out before any CVE.

Most CVEs are medium-to-low severity.

Every report is triaged with the researcher to agree on real scope and impact. It's the sheer **volume**, not the danger of any one finding, that makes this release worth your attention.

BEHIND EVERY ADVISORY

Every report gets an expert

STEP 01

Triaged by a committer

An expert on the very project it was reported against owns the issue.

STEP 02

Scope with the reporter

The team works directly with the researcher to confirm the real concern.

STEP 03

Reporter validates the fix

The researcher verifies the patch is correct before it ships.

For context: between Jan 2024 and Sep 2025, only **2.6%** of vulnerabilities reported to MITRE had a public proof of concept, making this level of reporter collaboration the exception, not the norm.

WHEN THE PATCH CADENCE SPEEDS UP, TIMING MATTERS

VMware Tanzu Spring can help

Day 0 access

Fixes land in the enterprise repository
before the public CVE is even announced.

First-party support

The only provider that can release a fix
ahead of public disclosure.

Application Advisor

Automated, real code upgrades as pull
requests in your CI, not just dependency
bumps.

New times, steady hands.

The volume of reports won't return to historic norms soon, but the process protecting you hasn't changed. Stay patched, and stay calm.

SPRING ENTERPRISE

enterprise.spring.io

RELEASE SCHEDULE

calendar.spring.io

RESOURCES

Where to go next

Spring Framework 7.0 release notes	spring-projects/spring-framework/wiki
Spring Boot 4.0 release notes	spring-projects/spring-boot/wiki
Road to GA blog series	spring.io/blog/2025/09/02/road_to_ga_introduction
Demo repository	github.com/danvega/sb4
Spring Portfolio Version Mappings	spring.io/projects/generations
Spring Release Highlights	spring.io/projects/release-highlights

How we got to Framework 7

Framework 6.2 Nov 2024

- Final 6.x with long-term support
- Foundation for Boot 3.4 & 3.5
- JDK 17 & JDK 21 LTS
- Deep core container revision

Framework 7.0 Nov 2025

- Foundation for Spring Boot 4.0+
- JDK 17+, optimized for JDK 25
- Jakarta EE 11, JSpecify, Kotlin 2.x
- Bean registration, API versioning

THANKS FOR WATCHING

Thank you!

dan.vega@broadcom.com

danvega.dev

[@therealdanvega](https://twitter.com/therealdanvega)